ASSIGNMENT 1: CREATE A README FILE FOR THE INGESTION UI DEVELOPED IN RAG

The README file was generated using the Copilot plugin for VS CODE. It describes the application's architecture and what each component does. Below is the content of the README file

 # RAG Mongo Demo

A demo RAG (Retrieval-Augmented Generation) pipeline for managing test cases and user stories. It lets you upload Excel exports, convert them to JSON, generate vector embeddings with Mistral AI, store everything in MongoDB Atlas, and then query it back using vector search, BM25 keyword search, hybrid search, and LLM-based reranking (Groq).

The project is a classic two-process local dev setup:

- **`client/`** — a Create React App (React 19 + MUI) single-page app, served on `http://localhost:3000`
- **`server/`** — an Express API (`server/index.js`), served on `http://localhost:3001`

The two only talk to each other over HTTP — the client is a pure REST client with a hardcoded `API_BASE = 'http://localhost:3001/api'` in every component (see `client/src/components/**`). There is no server-side rendering and no shared code between client and server at runtime.

```
Browser (React SPA, :3000)
    |  axios / fetch → http://localhost:3001/api/*
    ▼
Express API (server/index.js, :3001)
    |  spawns short-lived Node scripts, calls Mistral/Groq, talks to MongoDB
    ▼
MongoDB Atlas (vector store)   Mistral AI (embeddings)   Groq (rerank/summarize)
```

Running it

```bash
npm install       # installs server deps, postinstall installs client deps too
npm run dev       # concurrently runs `node server/index.js` (:3001) and `react-scripts start` (:3000)
```

Configuration lives in `.env` at the project root (see `.env.example`): MongoDB connection string, database/collection/index names, and the Mistral/Groq API keys. The `/api/env` endpoint (used by the **Settings** tab) can read and overwrite this file at runtime.

How the frontend is organized

`client/src/App.js` renders a fixed sidebar of tabs (no client-side routing — just component swapping via `useState`), each backed by one component in `client/src/components/`:

Tab	Component	Talks to
Convert to JSON	`data/ConvertToJson.js`	`POST /api/upload-excel`

Embeddings & Store	`data/EmbeddingsStore.js`	`GET /api/files`, `POST /api/create-embeddings-batch`, `GET /api/jobs/:id`
Query Preprocessing	`processing/QueryPreprocessing.js`	`POST /api/search/preprocess`, `/analyze`, `/deduplicate`
Vector Search	`search/QuerySearch.js`	`POST /api/search`
BM25 Search	`search/BM25Search.js`	`POST /api/search/bm25`
Hybrid Search	`search/HybridSearch.js`	`POST /api/search/hybrid`
Score Fusion	`search/RerankingSearch.js`	`POST /api/search/rerank`
Summarize & Dedup	`processing/SummarizationDedup.js`	`POST /api/search/summarize`
Prompt & Schema	`processing/PromptSchemaManager.js`	`POST /api/test-prompt`
Settings	`settings/Settings.js`	`GET/POST /api/env`

Every component does its own `axios`/`fetch` calls directly — there's no shared API client or global state manager (no Redux/Context for server data). Each component owns its own `useState` for loading/error/result, and uses `notistack` for toast notifications.

The core pipeline: Excel → JSON → Embeddings → MongoDB

This is the flow the "Convert to JSON" and "Embeddings & Store" tabs implement together, and it's the heart of the app.

1. Upload & convert (`ConvertToJson.js` → `POST /api/upload-excel`)

1. The user picks an `.xlsx`/`.xls` file, a ****Data Type**** (`testcases` or `userstories`), and a sheet name in the UI.
2. On submit, the client builds a `multipart/form-data` request (`file`, `sheetName`, `dataType`) and posts it to `POST /api/upload-excel`.
3. On the server (`server/index.js:271`), `multer` saves the upload to `uploads/` with a timestamped filename.
4. The handler ****writes a throwaway Node script to disk**** (`temp-excel-convert-*.js`) that uses the `xlsx` package to:
   - read the requested sheet (falling back to the first sheet if the name doesn't match),
   - map raw Excel column headers to normalized JSON keys via a `columnMap` (different maps for test cases vs. user stories — e.g. `"Test ID" → "id"`, `"Story Points" → "storyPoints"`),
   - write the result as an array of JSON objects to `src/data/converted-<timestamp>.json` (test cases) or `src/data/stories-<timestamp>.json` (user stories).
5. The server `spawn()`s that script as a child process, streams its stdout/stderr, waits for it to exit (30s timeout), then deletes both the temp script and the original upload.
6. The response (`{ success, outputFile, output }`) is shown back in the UI's "Conversion Results" card, and the new file now appears in `src/data/` ready for the next step.

Running the conversion as a spawned child script (rather than requiring it inline) is a deliberate isolation choice: it keeps the long-running `xlsx` parsing off the main Express event loop and gives each conversion a clean, disposable process.

2. Generate embeddings & ingest into MongoDB (`EmbeddingsStore.js`)

1. On mount, the tab calls `GET /api/files` to list every `*.json` file in `src/data/` (the outputs from step 1), and `GET /api/jobs/active` to resume any embedding job that was already running (e.g. after a page refresh).
2. The user selects one or more files in the MUI `DataGrid` and clicks **Create Embeddings**. The client auto-detects the job type from the filename (`testcase`/`test-case` → test cases, `stor`/`story` → user stories) and picks the matching batch script name (`create-embeddings-batch-mistral.js` or `create-userstories-embeddings-batch-mistral.js`).

3. It posts `{ files, scriptName, jobType }` to `POST /api/create-embeddings-batch`.
4. The server (`server/index.js:776`) first validates that the target Mongo
   database/collection exist, then immediately creates an in-memory **job**
   (`jobs` Map, `server/index.js:50`) and returns `{ jobId }` — the actual work
   happens asynchronously so the HTTP request doesn't hang for minutes.
5. `processBatchEmbeddings()` spawns the chosen script from
   `src/scripts/embeddings/`, passing the selected filenames via the
   `EMBEDDING_INPUT_FILES` env var. Each script (see
   `src/scripts/embeddings/create-embeddings-batch-mistral.js`):
   - reads the input JSON file(s),
   - chunks records into batches of 50 and calls **Mistral AI's embeddings API**
     (`src/scripts/utilities/mistralEmbedding.js`, model `mistral-embed`, 1024-dim
     vectors) with up to 3 concurrent batch calls (via `p-limit`) and retry/backoff,
   - attaches the resulting vector plus `embeddingMetadata` (model, cost, tokens)
     to each record,
   - inserts the enriched documents into MongoDB in sub-batches of 100 via
     `collection.insertMany()`.
6. As the child process writes to stdout, the server updates the job's `status`,
   `progress`, and `currentFile` in the `jobs` Map.
7. The client polls `GET /api/jobs/:jobId` every 2 seconds and renders a live
   progress bar until `status === 'completed'`, then shows a success/failure
   summary per file.

At the end of this pipeline, every test case / user story is a MongoDB document
shaped roughly like:

```jsonc
{
  "module": "...", "id": "...", "title": "...", "description": "...", "steps": "...",
  // ...other original fields...
  "embedding": [0.0123, -0.0456, /* 1024 floats */],
  "embeddingMetadata": { "model": "mistral-embed", "provider": "mistral-ai", "cost": 0.000004,
"tokens": 42 },
  "sourceFile": "converted-1234567890.json",
  "createdAt": "2026-...Z"
}
```

which is exactly what the vector search endpoint queries against.

3. Querying it back

MongoDB Atlas needs a **Vector Search index** (defined in
`src/config/testcases-vector-index.json` — cosine similarity over the 1024-dim
`embedding` field, plus filter fields for `id`/`module`/`title`/etc.) and, for
keyword search, a **BM25/text index** — these are created once in Atlas and
referenced by name via `VECTOR_INDEX_NAME` / `BM25_INDEX_NAME` in `.env`.

- **Vector Search** (`QuerySearch.js` → `POST /api/search`): embeds the query text
  with Mistral, then runs a `$vectorSearch` aggregation pipeline
  (`numCandidates` ≈ 10× the requested limit, cosine similarity), optionally
  post-filtered by metadata (`module`, `priority`, etc.) via `$match`.
- **BM25 Search** (`BM25Search.js` → `POST /api/search/bm25`): pure keyword/full-text
  search using Atlas Search's text index — no embedding call needed.
- **Hybrid Search** (`HybridSearch.js` → `POST /api/search/hybrid`): combines vector
  and BM25 results.
- **Score Fusion / Reranking** (`RerankingSearch.js` → `POST /api/search/rerank`):
  fuses BM25 + vector scores and/or reranks candidates using Groq
  (`src/scripts/utilities/groqClient.js`, `rerankDocuments()`).

- **Summarize & Dedup** (`SummarizationDedup.js` → `POST /api/search/summarize`, `/api/search/deduplicate`): also backed by Groq (`summarizeResults()`), used to collapse near-duplicate results and produce a natural-language summary.
- **Query Preprocessing** (`QueryPreprocessing.js` → `POST /api/search/preprocess`, `/analyze`): normalizes, expands abbreviations, and expands synonyms in the raw query before it's embedded (`src/scripts/query-preprocessing/`).

API reference (server/index.js)

Method & Path	Purpose
`GET /api/health`	Liveness check
`GET /api/jobs/active` / `GET /api/jobs/:jobId`	Poll background job status
`GET /api/files`	List convertible JSON files in `src/data/`
`GET /api/metadata/distinct`	Distinct `module`/`priority`/`risk`/`type` values for search filter dropdowns
`POST /api/upload-excel`	Upload Excel → convert to JSON (step 1)
`POST /api/create-embeddings`	Legacy per-file embedding (single Mistral call per record)
`POST /api/create-embeddings-batch`	Batch embedding + Mongo ingestion (step 2, used by the UI)
`GET /api/env` / `POST /api/env`	Read/write `.env` (Settings tab)
`POST /api/search`	Vector search
`POST /api/search/bm25`	Keyword/BM25 search
`POST /api/search/hybrid`	Combined vector + BM25
`POST /api/search/rerank`	Score-fusion + Groq reranking
`POST /api/search/preprocess`, `/analyze`	Query normalization/expansion
`POST /api/search/deduplicate`, `/summarize`	Groq-based dedup & summarization
`POST /api/test-prompt`	Prompt/schema testing (Prompt & Schema tab)
`POST /api/search/user-stories`	Search over the user-stories collection
`GET /api/testcases/latest-id`	Next available test case ID

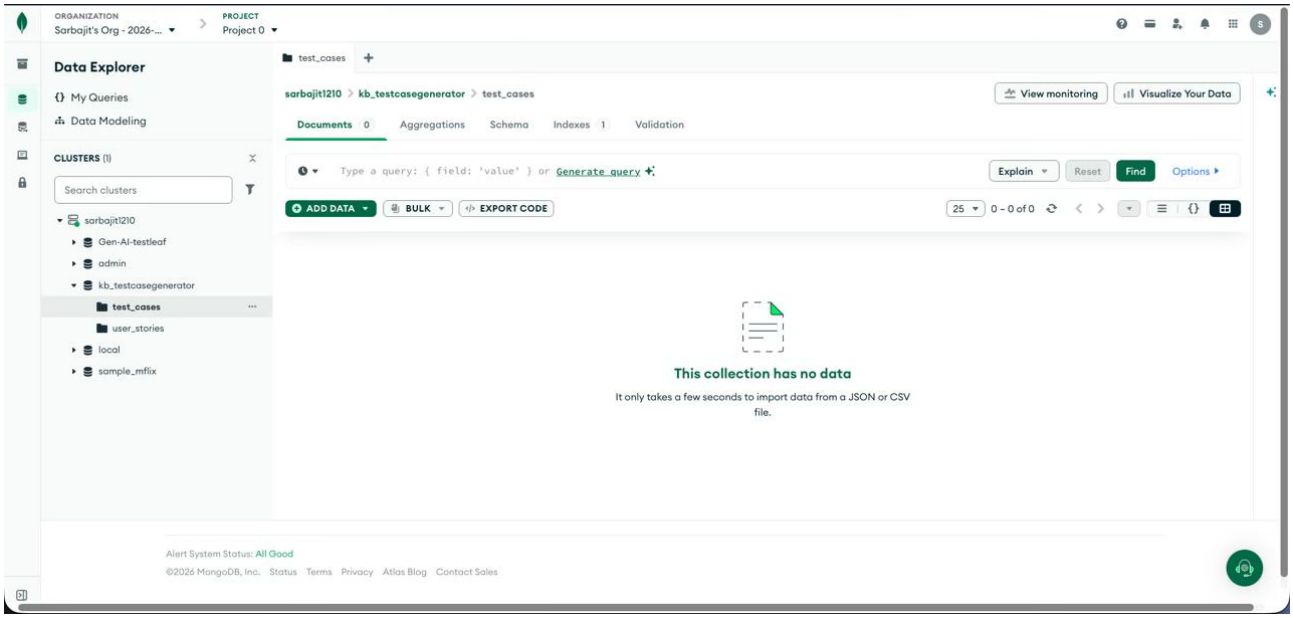
Notes

- Both the Excel→JSON conversion and the embedding/ingestion steps run as spawned child Node processes rather than inline in the Express handler, so a slow parse or a long embedding run can't block other API requests.
- Embedding jobs are tracked in an in-memory `Map` (`jobs`), not a database — job history is lost on server restart, though an in-progress job survives a **browser** refresh (the client re-polls `GET /api/jobs/active`).
- `.env` contains live credentials (MongoDB URI, Mistral/Groq API keys) and is not committed to git — copy `.env.example` and fill in your own values.

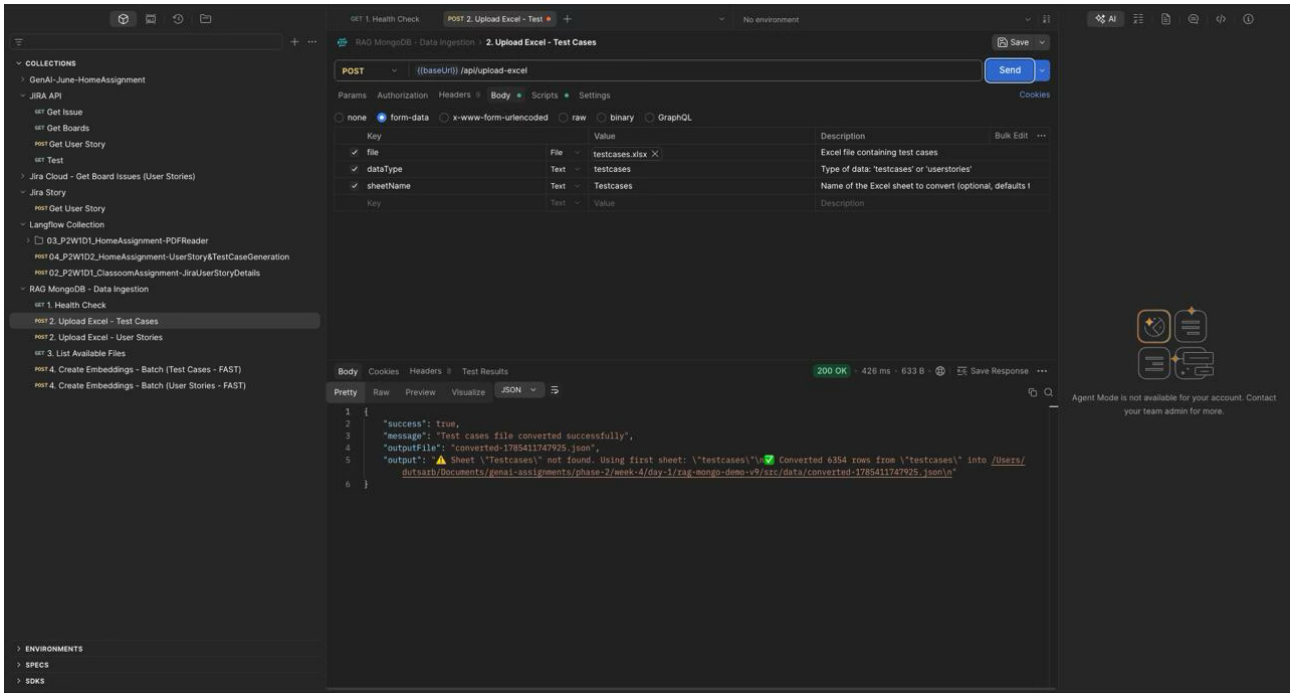
ASSIGNMENT 2: EXECUTE INGESTION PIPELINE USING POSTMAN API COLLECTION. DATA INGESTION INTO *test_cases* AND *user_stories* COLLECTION

1. INGESTION TO *test_cases*

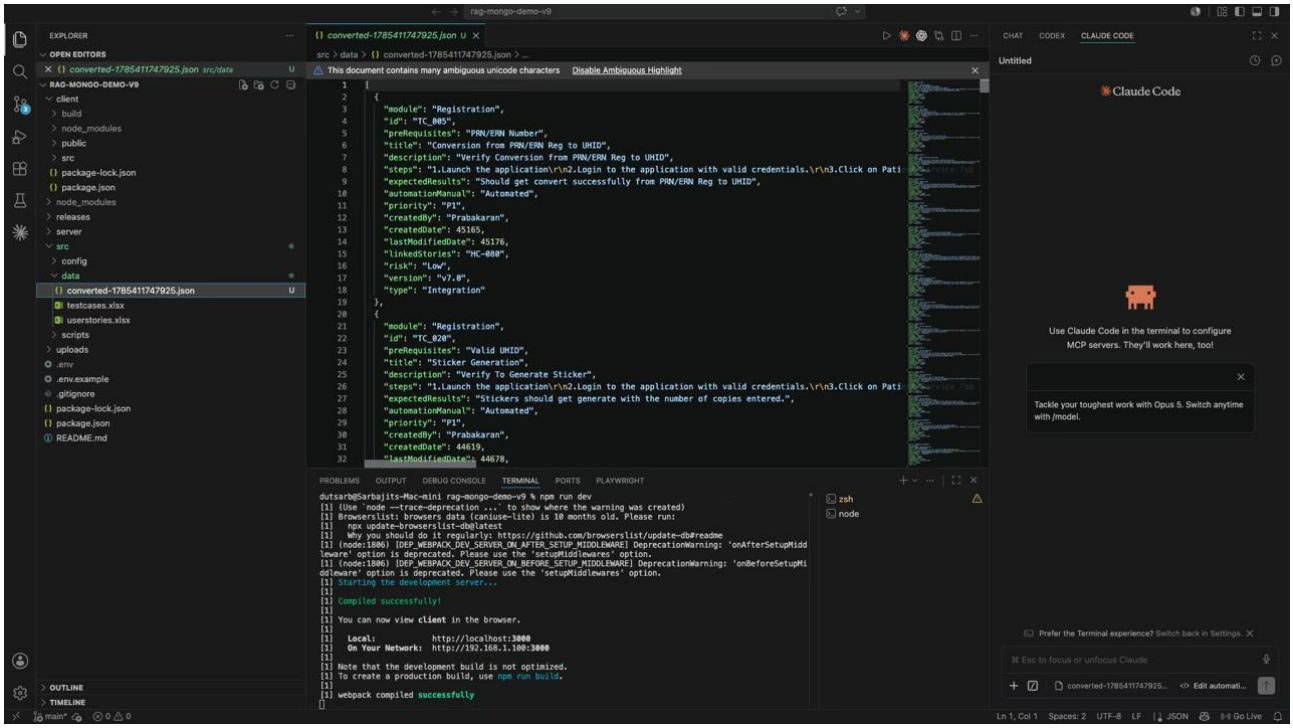
Step 1: No Existing data available under the test_cases collection



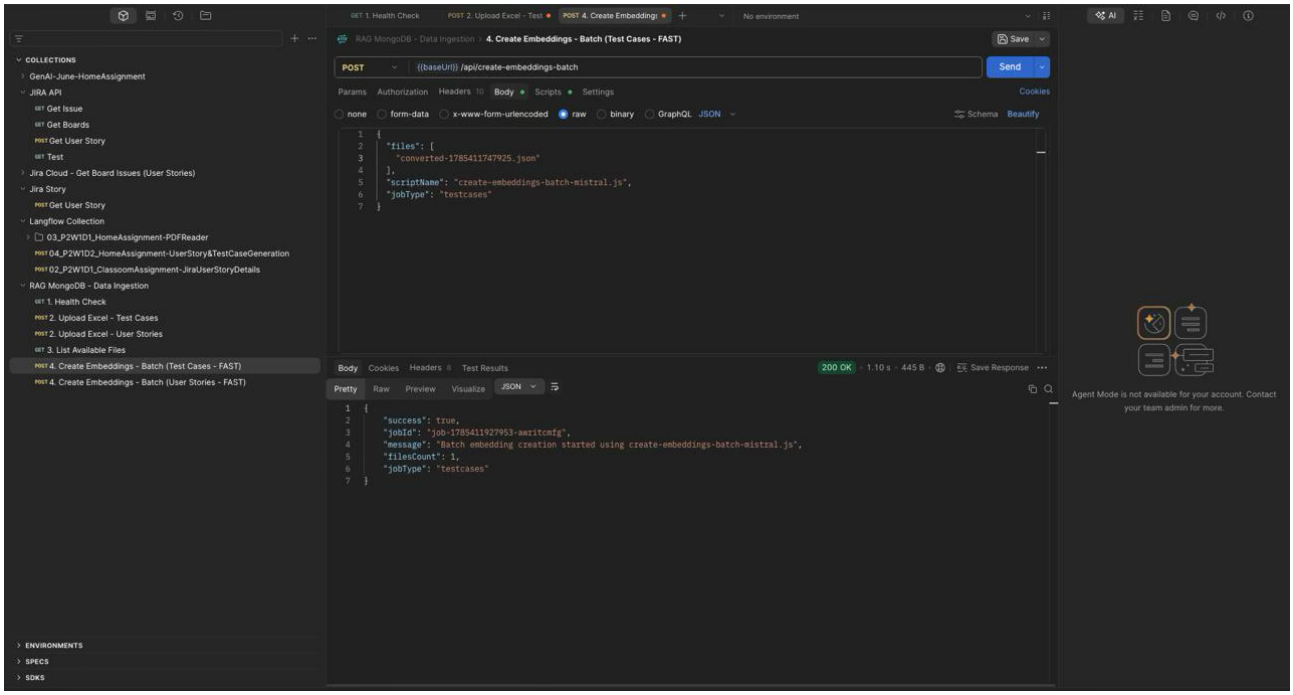
Step 2: Executed Upload Excel - Test Cases api to convert Excel test cases to JSON format



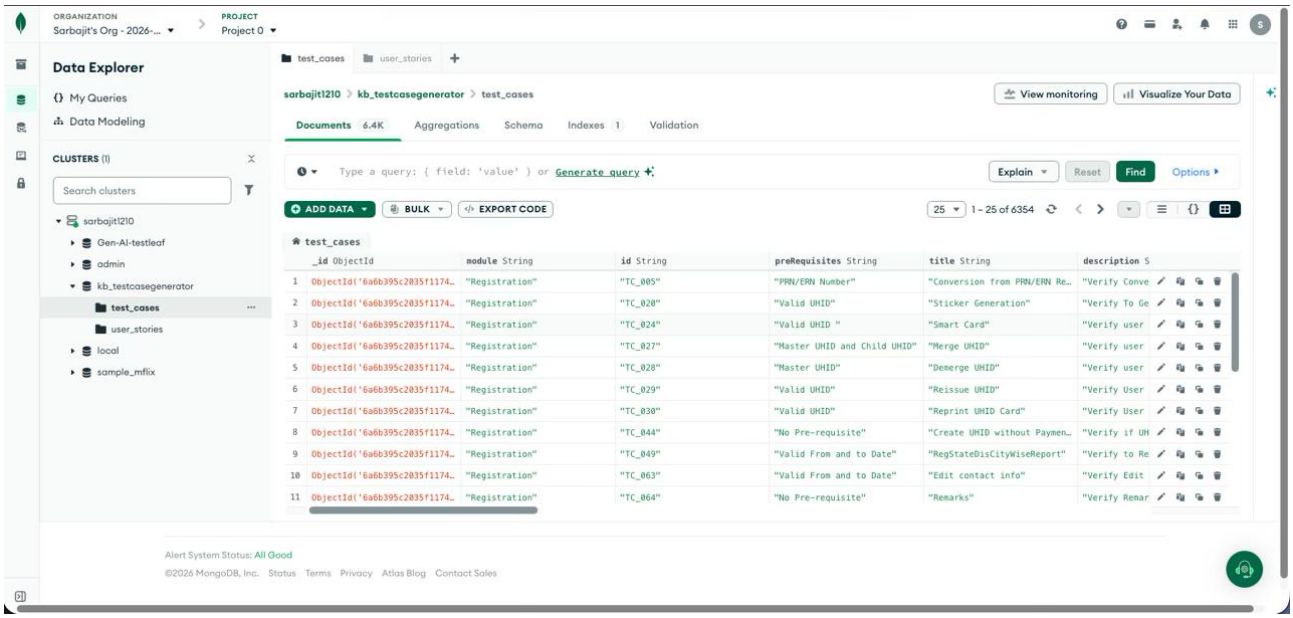
Step 3: A JSON file was created within the src->data folder within the project folder



Step 4: Executed Create Embeddings - Batch (Test Cases - FAST) api to initiate data upload to the test_cases collection

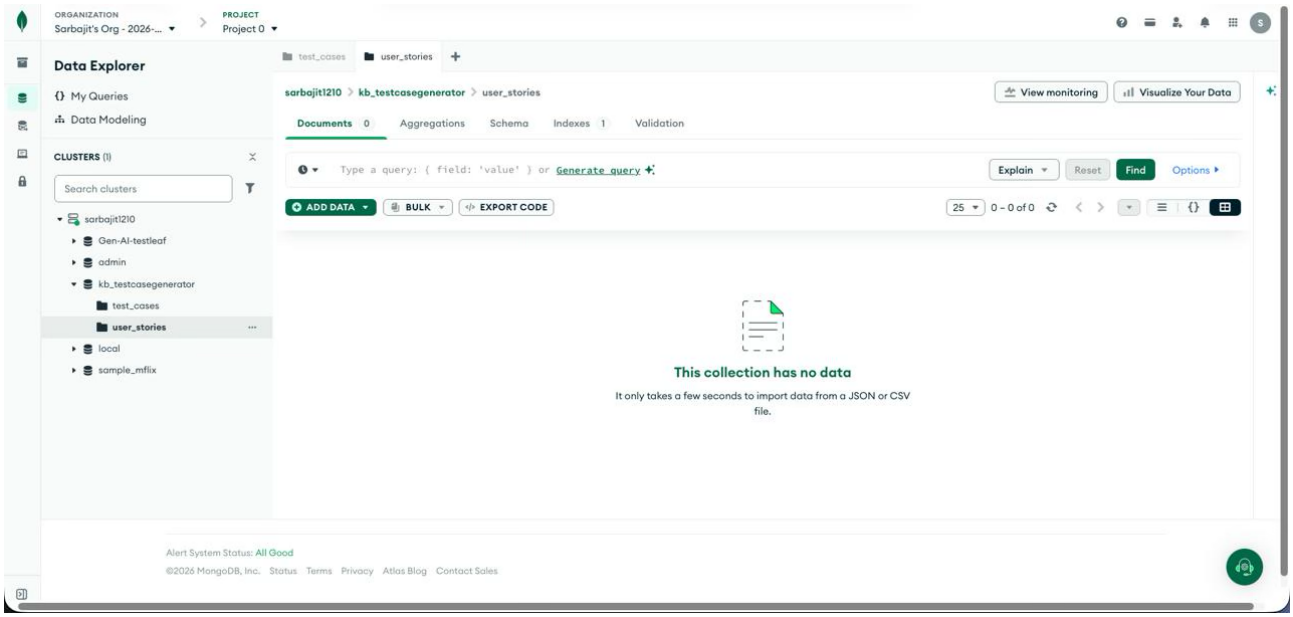


Step 5: The records are uploaded to the test_cases collection

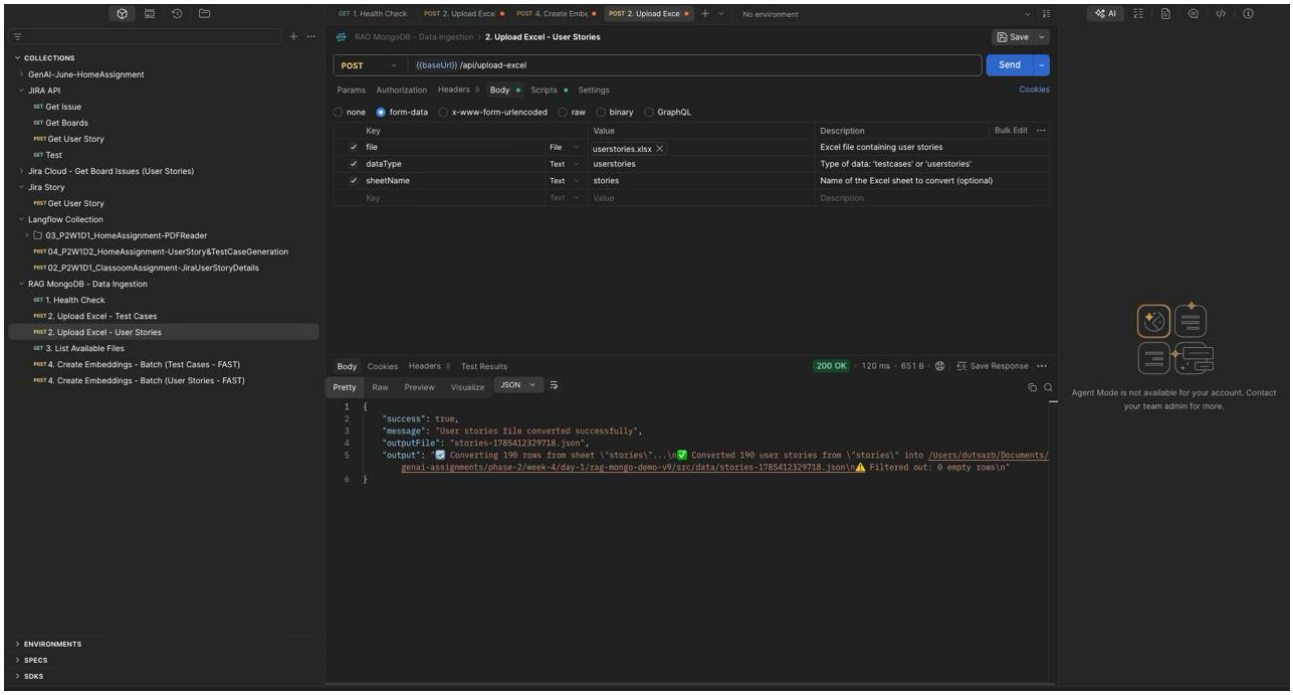


2. INGESTION TO *user_stories*

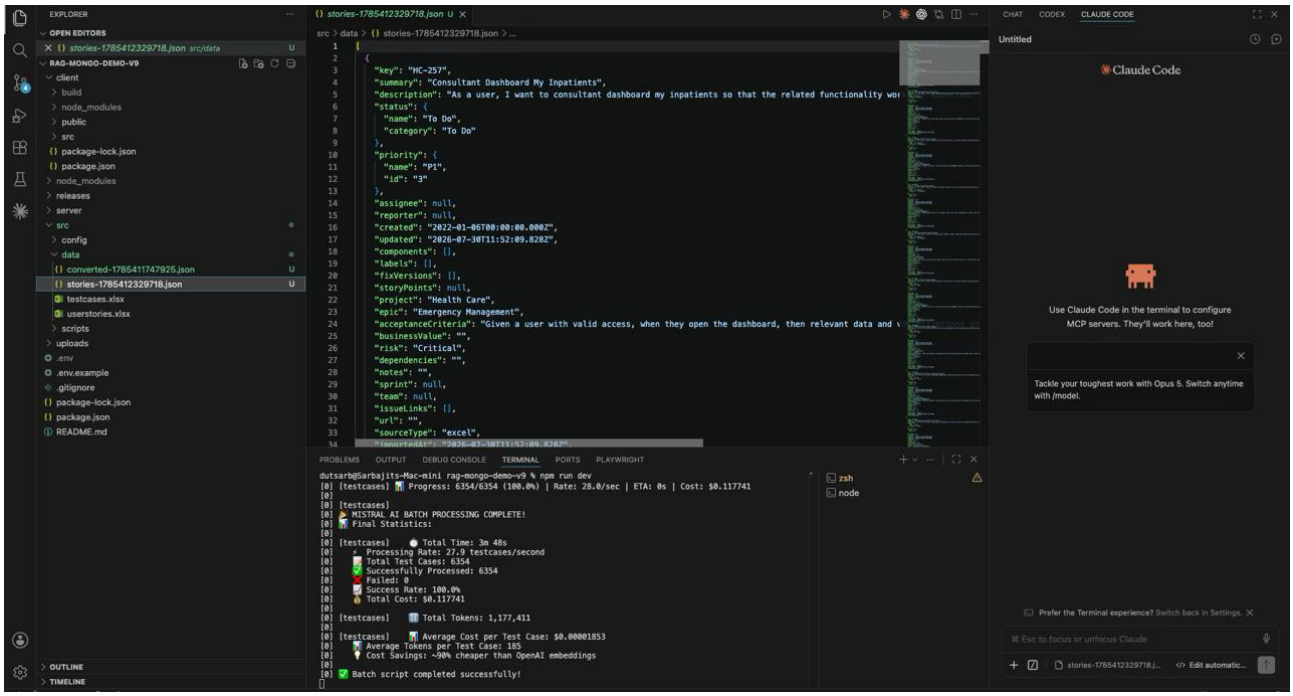
Step 1: No existing data available under the user_stories collection



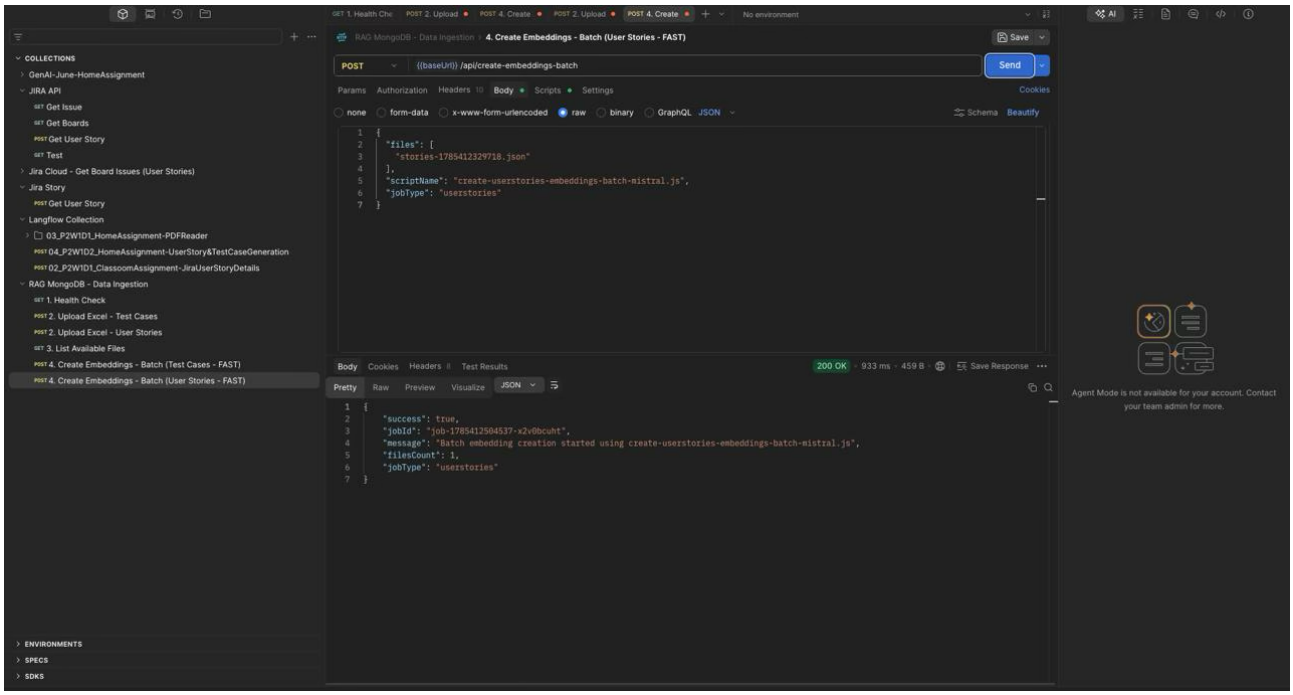
Step 2: Executed Upload Excel - User Stories api, to convert Excel user stories to JSON format



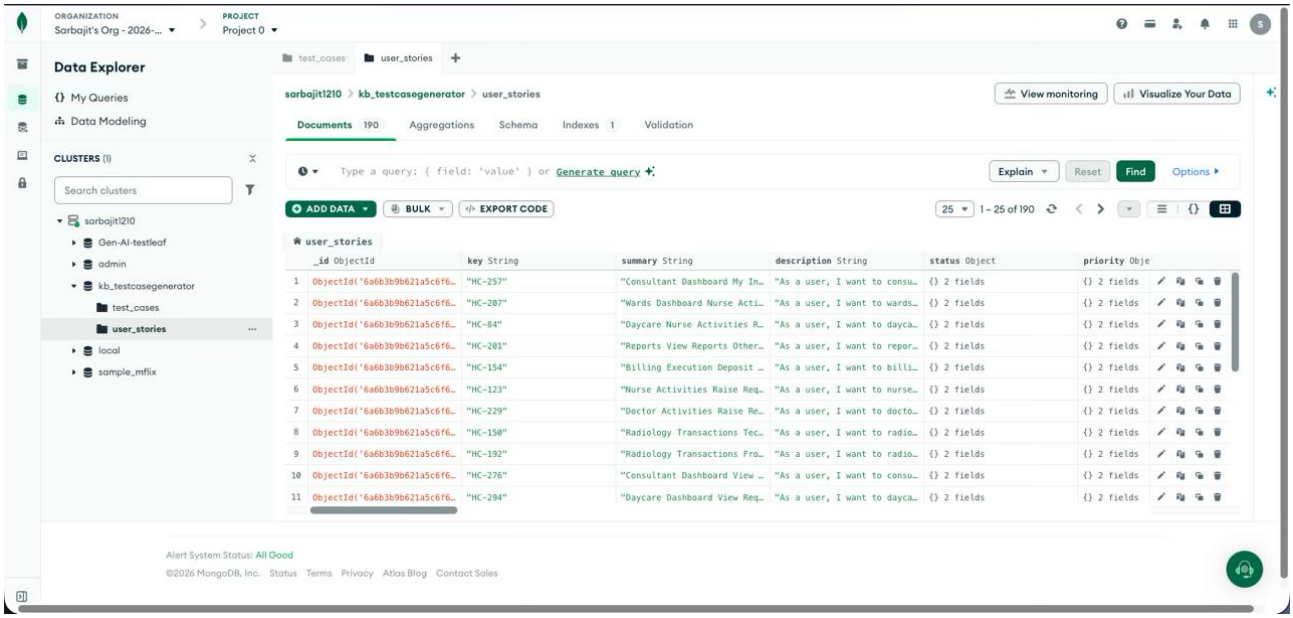
Step 3: A JSON file got created within the src->data folder within the project folder



Step 4: Executed Create Embeddings - Batch (User Stories - FAST) api to initiate data upload to the user_stories collection



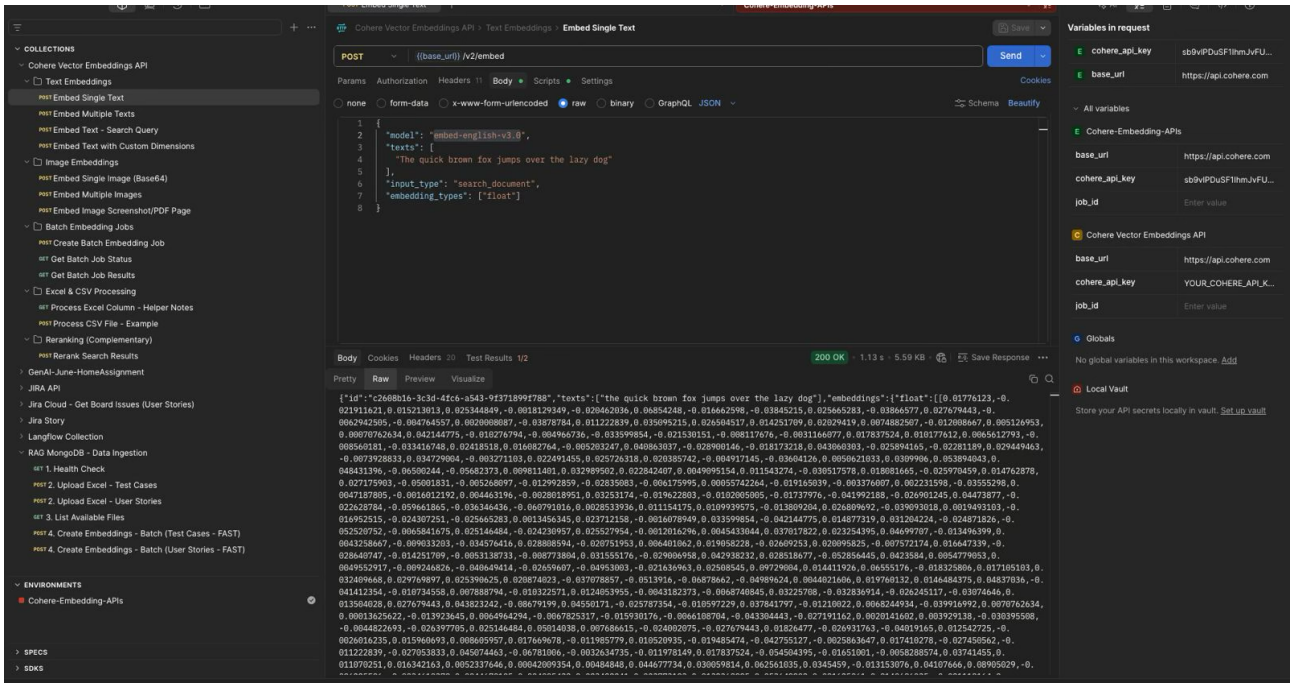
Step 5: The records are uploaded to the user_stories collection



ASSIGNMENT 3: EXPLORATORY TASK: VECTOR EMBEDDING USING COHERE AI

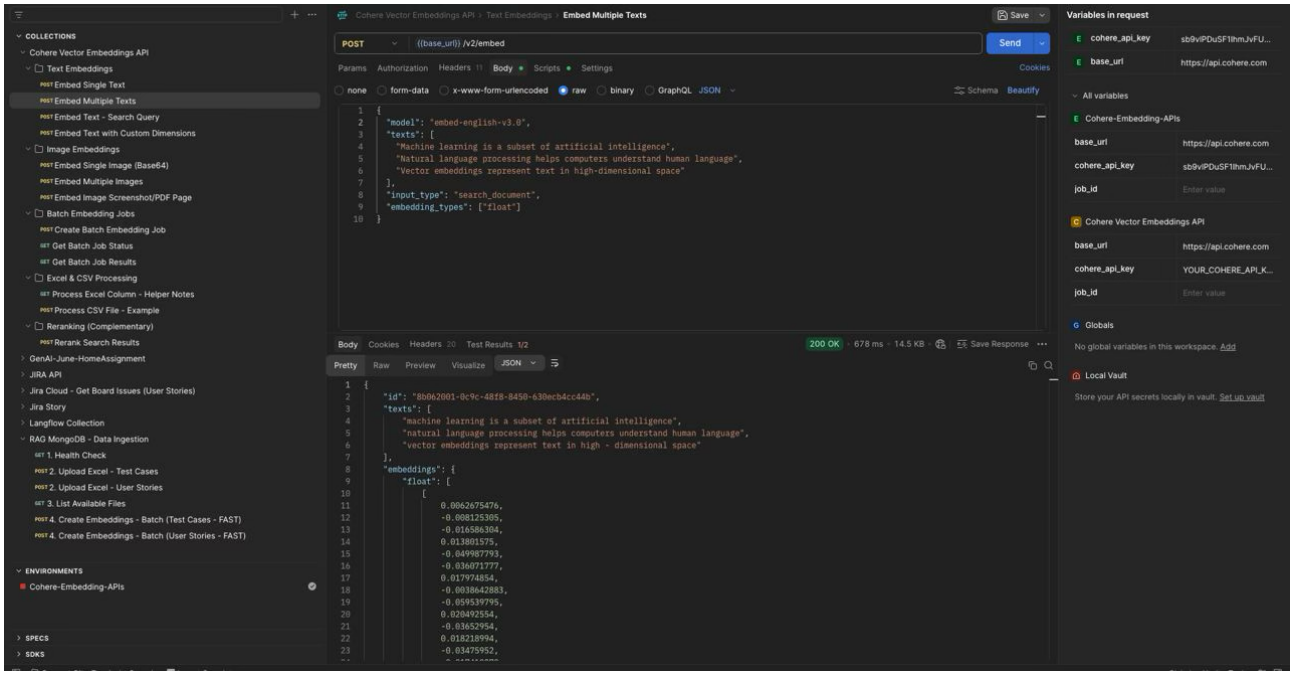
EMBEDDING OF SINGLE QUERY

Executed the Embed Single Text API using the embed-english-v3.0 model. This model can embed text and images with a dimension of 1024 and a max tokens (context length) of 512 tokens

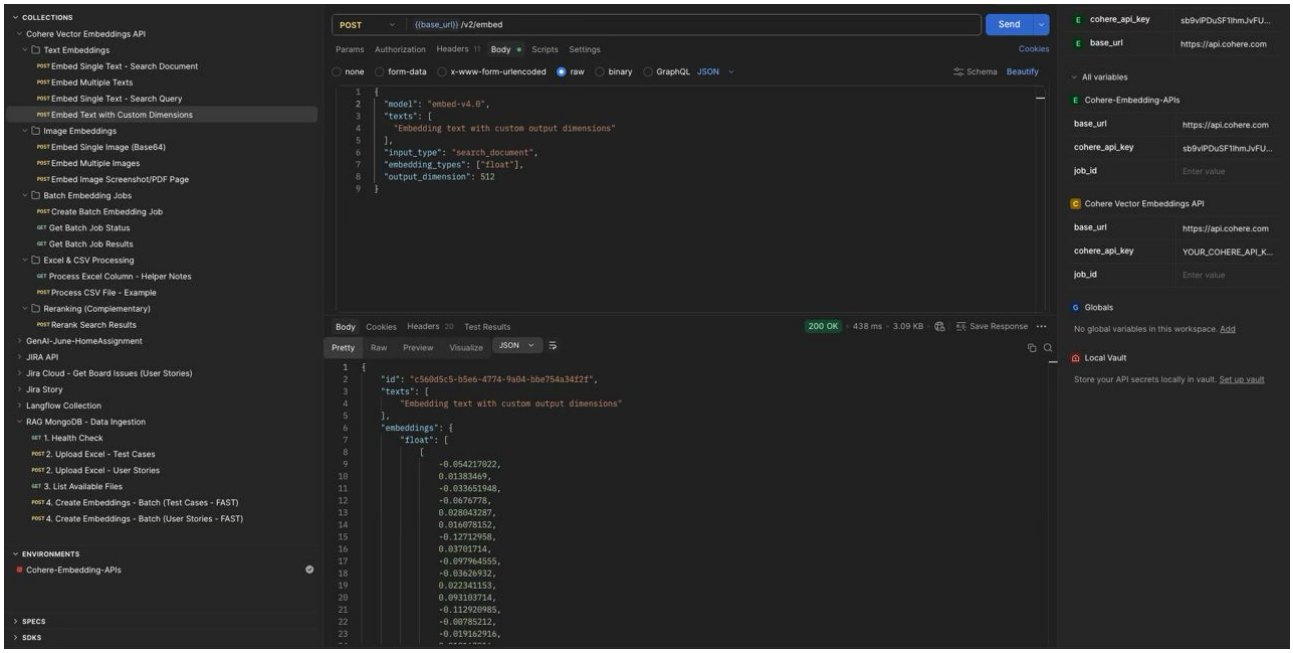


EMBEDDING OF MULTIPLE QUERIES

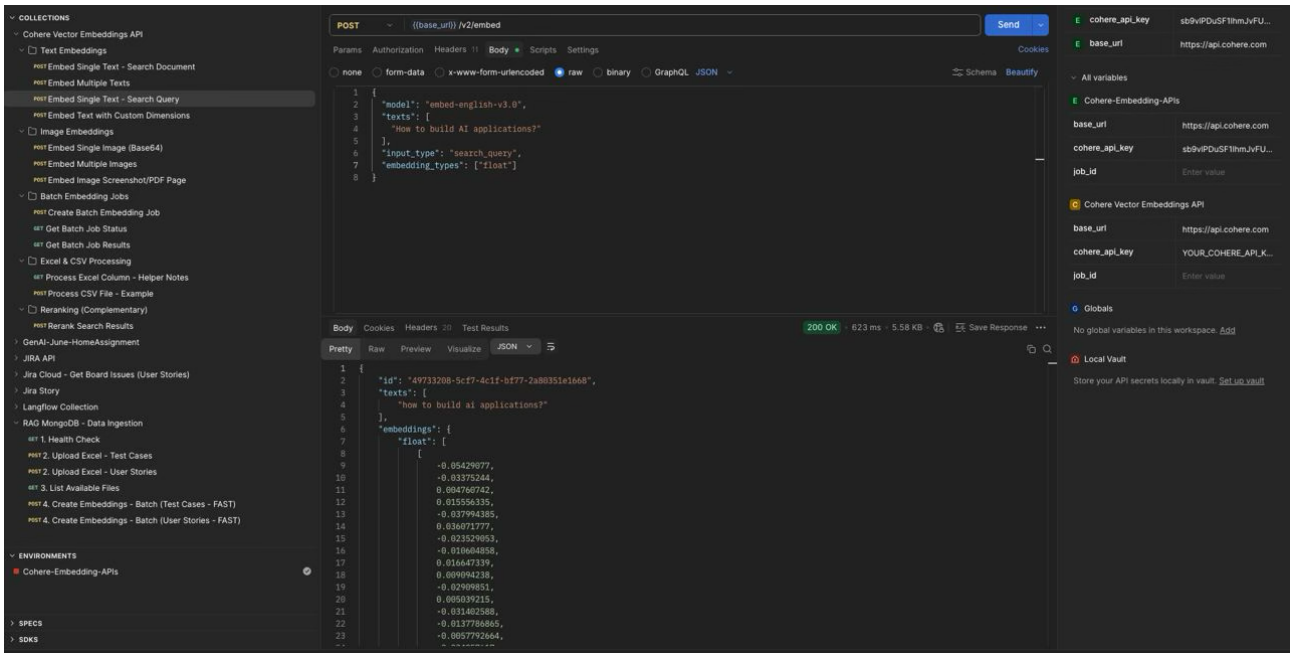
Executed the Embed Multiple Texts API to vector-embed multiple lines of text. The model used for this API call is embed-english-v3.0



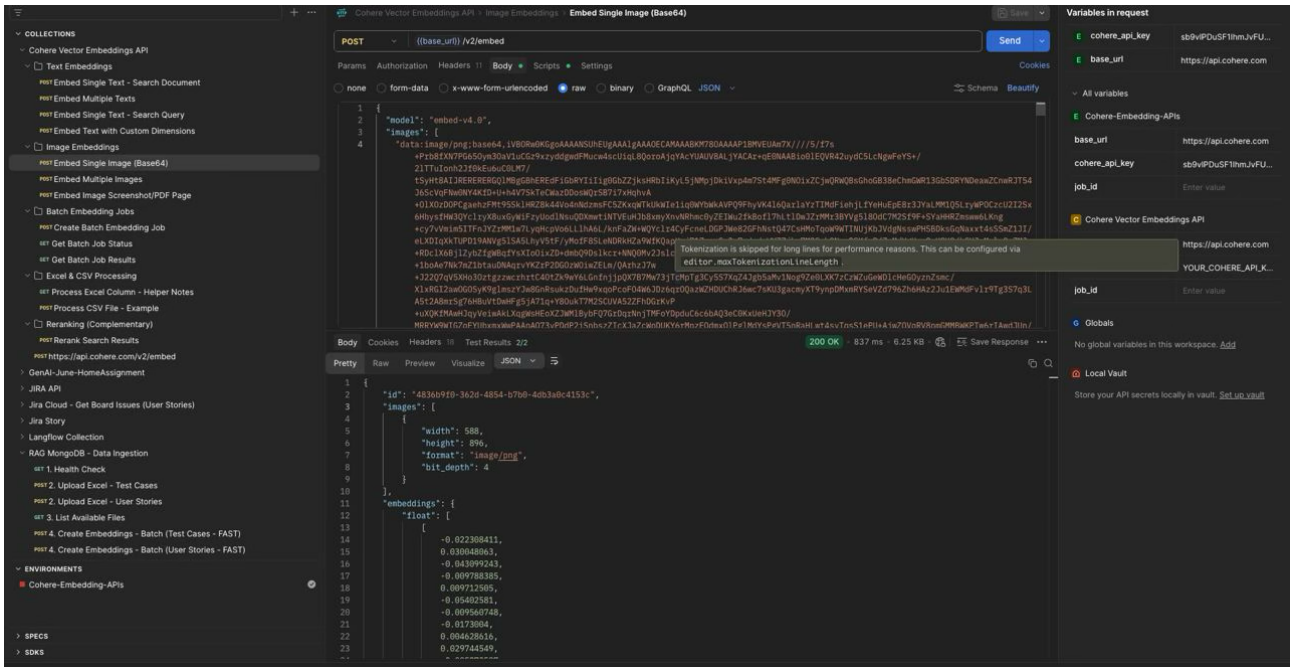
Embed a single line of text using search_query as the input_type field. The model used is embed-english-v3.0. This type of API call is done while embedding a search query made against an existing vector DB to perform a search



Embedding text with custom output dimension using “output_dimension” parameter. This parameter is applicable only for v4.0 models



The **Image Embedding** API for a single image is invoked by passing a Base64 URI of the image file. It is embedded using the embed-v4.0 model



Executing Embedding for multiple image files. The images are embedded into two vector embedding files, each having 1024 dimensions